

Documento tecnico Reto 2 - Sistema VuelosApp

Este documento actualiza la documentacion tecnica del proyecto para el Reto 2, correspondiente a las semanas 7 a 11 de Integracion de Sistemas. El alcance se basa en el enunciado del proyecto `Booking Prototipo` y en la implementacion actual del repositorio `Sistema_vuelos.v1`.

El sistema se trata como un sistema individual de venta y reserva de vuelos que debe integrarse con el Booking Prototipo. Para Reto 2, la arquitectura evoluciona desde una API REST centralizada hacia microservicios con API Gateway, contratos versionados, observabilidad, seguridad y una estrategia de pruebas de integracion.

1. Alcance del Reto 2

Objetivo del reto:

- Migrar la aplicacion a una arquitectura de microservicios.
- Exponer servicios desacoplados para integracion con Booking Prototipo.
- Mantener el marketplace web y el panel administrativo.
- Incorporar API Gateway, contratos versionados, pruebas, observabilidad y seguridad.
- Desplegar una version distribuida y defendible tecnicamente.

Estado actual del proyecto:

Entregable Reto 2	Estado en el repositorio
Arquitectura migrada a microservicios	Implementada mediante entypoints por servicio y <code>docker-compose.yml</code>
API Gateway configurado	Implementado en <code>api-gateway</code> con proxy REST, GraphQL, rate limit y circuit breaker
Contratos versionados	REST bajo <code>/api/v1</code> , GraphQL en <code>/graphql</code> , contratos gRPC en <code>src/grpc/proto</code>
Integracion servicio a servicio	Implementada entre Booking y Flights mediante cliente HTTP interno e <code>x-internal-api-key</code>
Observabilidad y trazabilidad	OpenTelemetry, Jaeger, logs JSON y <code>X-Correlation-Id</code>
Seguridad	JWT, roles, ownership, CORS, Helmet, rate limiting e internal API key
Evidencia de pruebas	Parcial. Existe un spec Angular, pero falla; no hay pruebas backend/gateway automatizadas

2. Arquitectura general

La arquitectura de Reto 2 usa un API Gateway como punto de entrada y varios servicios de dominio desplegados de forma independiente.

```

flowchart LR
    Web[Angular Marketplace/Admin] --> GW[API Gateway :3000]
    BookingCentral[Booking Prototipo] --> GW

    GW --> Auth[auth-service :3001]
    GW --> Catalog[catalog-service :3002]
    GW --> Flights[flights-service :3003]
    GW --> Booking[booking-service :3004]
    GW --> Payments[payments-service :3005]
    GW --> Admin[admin-service :3006]

    Flights -. gRPC .-> Grpc[gRPC :50051]

    Auth --> AuthDB[(Auth DB)]
    Catalog --> CatalogDB[(Catalog DB)]
    Flights --> FlightsDB[(Flights DB)]
    Booking --> BookingDB[(Booking DB)]
    Payments --> PaymentsDB[(Payments DB)]
    Admin --> AdminDB[(Admin/Audit DB)]

    GW --> Jaeger[Jaeger / OTEL]
    Auth --> Jaeger
    Catalog --> Jaeger
    Flights --> Jaeger
    Booking --> Jaeger
    Payments --> Jaeger
    Admin --> Jaeger

```

Decision arquitectonica:

- El frontend no consume directamente los microservicios; consume el API Gateway.
- REST/JSON sigue siendo el contrato publico principal para Angular y para integracion externa.
- GraphQL actua como agregador opcional para vistas que combinan varios servicios.
- gRPC queda disponible como contrato interno o para integracion con Booking Prototipo.
- El monolito `vuelos-backend/src/server.ts` permanece como compatibilidad, pero el despliegue de Reto 2 debe usar los endpoints de microservicios.

3. Inventario de microservicios

Servicio	Puerto	Entry point	Responsabilidad	Base de datos / schema
API Gateway	3000	<code>api-gateway/src/server.ts</code>	Entrada publica, proxy REST, GraphQL, seguridad perimetral, health agregado	No aplica
Auth Service	3001	<code>vuelos-backend/src/services/auth-service.ts</code>	Login, registro, validacion de token activo	<code>auth.schema.prisma</code>
Catalog Service	3002	<code>vuelos-backend/src/services/catalog-service.ts</code>	Países, ciudades, aeropuertos, aerolíneas, aeronaves, catálogos	<code>catalog.schema.prisma</code>
Flights Service	3003 / 50051	<code>vuelos-backend/src/services/flights-service.ts</code>	Vuelos, segmentos, clases, promociones, disponibilidad, gRPC	<code>flights.schema.prisma</code>
Booking Service	3004	<code>vuelos-backend/src/services/booking-service.ts</code>	Reservas, pasajeros, perfiles de facturación, boarding passes	<code>booking.schema.prisma</code>
Payments Service	3005	<code>vuelos-backend/src/services/payments-service.ts</code>	Pagos, facturas, items, servicios extra de pasajero	<code>payments.schema.prisma</code>
Admin Service	3006	<code>vuelos-backend/src/services/admin-service.ts</code>	Dashboard administrativo, usuarios y auditoria	<code>admin.schema.prisma</code> y clientes de lectura por dominio

El archivo `docker-compose.yml` orquesta:

- `api-gateway`
- `auth-service`
- `catalog-service`
- `flights-service`
- `booking-service`
- `payments-service`
- `admin-service`
- `jaeger`

Cada servicio backend se construye desde el mismo `Dockerfile` multi-stage, usando targets separados por servicio.

4. Contratos y compatibilidad - Semana 7

REST versionado

El contrato publico mantiene el prefijo `/api/v1`. El API Gateway enruta cada grupo de recursos al microservicio correspondiente.

Prefijo REST	Servicio destino
/api/v1/auth	auth-service
/api/v1/countries , /cities , /airports , /airlines , /aircraft , /service-catalog	catalog-service
/api/v1/flights , /flight-classes , /segments , /promotions	flights-service
/api/v1/reservations , /reservas , /reservation-passengers , /billing-profiles , /boarding-passes	booking-service
/api/v1/payments , /invoices , /invoice-items , /passenger-services	payments-service
/api/v1/admin , /audit-logs	admin-service

Formato de respuesta REST:

```
{
  "success": true,
  "data": {}
}
```

Formato de error:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "Descripcion del error"
  }
}
```

Compatibilidad:

- Se mantienen alias sin version para rutas usadas por frontend o integraciones previas, por ejemplo /api/flights , /api/auth , /api/reservations .
- El Gateway centraliza la tabla de enrutamiento en api-gateway/src/config/registry.ts .
- El contrato OpenAPI del backend se conserva en /api/v1/docs , /api/v1/docs.json y /api/v1/spec .

GraphQL aggregator

El Gateway expone /graphql con GraphQL Yoga.

Consultas y mutaciones principales:

Operacion GraphQL	Servicios usados	Proposito
flightSearch	flights-service y catalog-service	Agregar vuelos, rutas, aeropuertos, aerolineas y clases
reservation(id)	booking-service y payments-service	Consultar detalle de reserva con pagos y boarding passes
myReservations	booking-service	Consultar reservas del usuario autenticado
assignSeat	booking-service	Asignar asiento
addAncillary	payments-service	Agregar servicio extra

Nota tecnica: GraphQL propaga el JWT hacia los microservicios. Los servicios siguen siendo responsables de validar token, rol y ownership. La consulta myReservations debe alinearse con el endpoint real /api/v1/reservations/my , porque el resolver actual llama a /api/v1/reservations?userId=... , que en el router REST esta protegido para administradores.

gRPC y Protobuf

Existen contratos .proto en vuelos-backend/src/grpc/proto :

- flights.proto

- `reservations.proto`
- `passengers.proto`
- `payments.proto`
- `promotions.proto`
- `checkin.proto`

El servidor gRPC se levanta en el puerto `50051` y expone servicios como:

- `FlightService.SearchFlights`
- `FlightService.GetFlight`
- `FlightService.GetFlightAvailability`
- `ReservationService.CreateReservation`
- `ReservationService.CancelReservation`
- `PaymentService.ProcessPayment`
- `PromotionService.ValidatePromotion`
- `CheckInService.CheckIn`

Uso propuesto:

- REST para frontend y consumidores web.
- GraphQL para agregacion de vistas.
- gRPC para llamadas internas de alto rendimiento o integracion con Booking Prototipo.

5. API Gateway y enrutamiento - Semana 8

El API Gateway implementa el patron Gateway/BFF/Aggregator.

Responsabilidades:

- Exponer un unico punto de entrada HTTP.
- Enrutar prefijos REST hacia microservicios.
- Aplicar Helmet, CORS y rate limiting.
- Propagar `X-Correlation-Id`.
- Exponer `/health` con estado agregado de servicios.
- Exponer `/graphql` para consultas agregadas.
- Aplicar circuit breaker por servicio.

Flujo de una solicitud REST:

```
sequenceDiagram
    autonumber
    participant Cliente as Angular / Booking
    participant Gateway as API Gateway
    participant Servicio as Microservicio destino
    participant DB as Base de datos de dominio

    Cliente->>Gateway: GET /api/v1/flights/search
    Gateway->>Gateway: CORS, rate limit, correlation id
    Gateway->>Servicio: Proxy a flights-service
    Servicio->>DB: Consulta de vuelos y disponibilidad
    DB-->>Servicio: Datos
    Servicio-->>Gateway: { success, data }
    Gateway-->>Cliente: { success, data } + X-Correlation-Id
```

Circuit breaker:

- Umbral de apertura: 5 fallos consecutivos.
- Tiempo de recuperacion: 30 segundos.

- Estado visible en `/health` del Gateway.
- Respuesta durante circuito abierto: `503 SERVICE_UNAVAILABLE` con `Retry-After`.

Service Mesh:

- No se implementa un service mesh formal como Istio o Linkerd.
- Para el alcance actual, Docker network + API Gateway + OpenTelemetry cubren enrutamiento, trazabilidad y control basico.
- Si el sistema crece, el siguiente paso seria mover mTLS, retries, traffic shaping y policy enforcement a un service mesh.

6. Integracion entre servicios

Reserva con validacion de disponibilidad

El Booking Service no modifica directamente los datos del dominio de vuelos. Para cupos y promociones usa `FlightsServiceClient`.

```
sequenceDiagram
    autonumber
    actor Cliente
    participant GW as API Gateway
    participant Booking as booking-service
    participant Flights as flights-service
    participant BDB as Booking DB
    participant FDB as Flights DB
    participant EventBus as EventBus

    Cliente->>GW: POST /api/v1/reservations
    GW->>Booking: Proxy REST con JWT y correlation id
    Booking->>Flights: GET /api/v1/flight-classes/{id}
    Flights->>FDB: Leer clase y cupos
    Flights-->>Booking: Clase disponible
    Booking->>Flights: PATCH /internal/flight-classes/{id}/decrement-seats
    Flights->>FDB: Descontar cupos
    Booking->>BDB: Crear reserva y pasajeros
    Booking->>EventBus: ReservationCreated
    Booking-->>GW: 201 { success, data }
    GW-->>Cliente: Reserva creada
```

Proteccion servicio a servicio:

- Las rutas internas usan `x-internal-api-key`.
- El middleware `requireInternalService` rechaza llamadas sin clave o con clave invalida.
- La URL del flights-service se obtiene desde `FLIGHTS_SERVICE_URL`.

Pagos, facturas y extras

El Payments Service administra:

- `Payment`
- `Invoice`
- `InvoiceItem`
- `PassengerService`

Eventos publicados:

- `PaymentRegistered`
- `InvoiceIssued`
- `AncillaryAdded`

Estos eventos se publican actualmente en un bus in-process y se registran como logs estructurados para auditoria. En una evolucion posterior, el bus debe reemplazarse por RabbitMQ, Kafka, NATS o Azure Service Bus.

7. Observabilidad y trazabilidad - Semana 10

Componentes implementados:

Mecanismo	Implementacion
Correlation ID	Middleware en gateway y microservicios, header <code>X-Correlation-Id</code>
Logs estructurados	JSON con <code>ts</code> , <code>service/component</code> , <code>method</code> , <code>path</code> , <code>status</code> , <code>ms</code> , <code>cid</code>
Trazas distribuidas	OpenTelemetry Node SDK
Exportador	OTLP HTTP hacia Jaeger
UI de trazas	Jaeger en <code>http://localhost:16686</code>
Health checks	<code>/health</code> por servicio y agregado en Gateway

Ejemplo de log:

```
{
  "ts": "2026-05-26T16:00:00.000Z",
  "service": "booking-service",
  "method": "POST",
  "path": "/api/v1/reservations",
  "status": 201,
  "ms": 42,
  "cid": "1fbb9f38-..."
}
```

Estrategia de trazabilidad:

- El cliente puede enviar `x-correlation-id`; si no lo envía, el sistema genera uno.
- El Gateway propaga el header hacia servicios.
- Los microservicios devuelven `X-Correlation-Id`.
- Los eventos de dominio incluyen `correlationId`.
- Jaeger permite inspeccionar llamadas HTTP y latencias cuando los servicios arrancan con `--import=./dist/shared/telemetry/tracer.js`.

8. Seguridad en integraciones - Semana 11

Controles implementados:

Control	Implementacion
Autenticacion	JWT Bearer con <code>jsonwebtoken</code>
Configuracion JWT	<code>JWT_SECRET</code> , <code>JWT_ISSUER</code> , <code>JWT_AUDIENCE</code> , <code>JWT_EXPIRES_IN</code>
Validacion de token activo	Auth Service verifica <code>isActive</code> y <code>tokenVersion</code>
Autorizacion	Roles <code>ADMIN</code> y usuario autenticado
Ownership	Middlewares para reservas, pasajeros, pagos, facturas, boarding passes
Seguridad perimetral	Helmet, CORS allowlist y body limit de 1 MB
Rate limiting	Global, auth y busqueda
Servicio a servicio	<code>x-internal-api-key</code> en rutas <code>/internal/*</code>
Auditoria	<code>audit-logs</code> y eventos de dominio publicados/logueados

Rutas protegidas:

- Mutaciones administrativas requieren `ADMIN`.
- Operaciones de cliente requieren JWT.
- Operaciones sensibles validan ownership antes de devolver o modificar datos.
- Pagos de cliente no confían en el monto enviado por el frontend; se recalcula desde la reserva.
- Asignación de asiento valida que la reserva pertenezca al usuario y que el asiento no este ocupado.

Riesgos actuales:

- El archivo `.env.example` raíz aun no lista todas las variables de microservicios que usa `docker-compose.yml`, como `AUTH_DATABASE_URL`, `CATALOG_DATABASE_URL`, `FLIGHTS_DATABASE_URL`, `BOOKING_DATABASE_URL`, `PAYMENTS_DATABASE_URL`, `ADMIN_DATABASE_URL` e `INTERNAL_API_KEY`.
- GraphQL decodifica el `userId` del JWT para contexto, pero la validación real ocurre al pasar el token a los microservicios. Conviene ajustar resolvers para no depender de claims no verificados antes de llamar a rutas protegidas.

9. Estrategia de pruebas - Semana 9

Estado actual verificado

Revision ejecutada el 2026-05-26:

- Búsqueda de archivos `*.spec.ts` y `*.test.ts`: solo se encontro `vuelos-angular/src/app/app.spec.ts`.
- `vuelos-angular/package.json` tiene script `test`.
- `vuelos-backend/package.json` no tiene script `test`.
- `api-gateway/package.json` no tiene script `test`.
- `vuelos-backend/test-db.ts` es un script manual de consulta a Prisma, no una suite automatizada.

Resultado de ejecucion:

```
Comando: npm.cmd test -- --watch=false
Proyecto: vuelos-angular
Resultado: 1 archivo de prueba, 2 tests, 1 pasa y 1 falla
Falla: src/app/app.spec.ts:21 espera un h1 con "Hello, vuelos-angular"
Causa: app.html actualmente solo contiene <router-outlet />
```

Conclusion:

- Si hay test frontend, pero es el spec base generado por Angular y esta desactualizado.
- No hay evidencia automatizada de pruebas de integración entre microservicios.
- No hay contract tests provider-consumer.
- No hay pruebas backend ni gateway registradas en scripts de npm.

Matriz minima recomendada para Reto 2

Tipo de prueba	Objetivo	Herramienta sugerida	Estado
Unit frontend	Componentes, guards, services Angular	Vitest + Angular Testing	Parcial, falla spec base
Unit backend	Servicios de dominio sin Express	Vitest o Jest	Pendiente
API integration	Endpoints REST con app Express	Vitest/Jest + Supertest	Pendiente
Contract testing	Contrato Booking consumidor vs Vuelos proveedor	Pact o Schemathesis/OpenAPI	Pendiente
Gateway routing	Prefijos REST enrutan al servicio correcto	Supertest + mocks HTTP	Pendiente
Security tests	JWT, roles, ownership e internal key	Supertest	Pendiente
GraphQL tests	<code>flightSearch</code> , <code>reservation</code> , <code>assignSeat</code>	GraphQL Yoga test client	Pendiente
gRPC tests	RPCs de <code>flights.proto</code> y <code>reservations.proto</code>	<code>@grpc/grpc-js</code> test client	Pendiente

Casos prioritarios

1. POST /api/v1/auth/register crea usuario sin permitir escalar rol desde el cliente. 2. POST /api/v1/auth/login devuelve JWT valido. 3. GET /api/v1/flights/search responde vuelos disponibles. 4. POST /api/v1/reservations crea reserva y descuenta cupos en flights-service. 5. PATCH /api/v1/reservations/:id/cancel cancela y devuelve cupos. 6. PATCH /api/v1/reservations/:id/passengers/:passengerId/seat evita doble asignacion. 7. POST /api/v1/payments usa el monto real de la reserva. 8. GET /api/v1/admin/* rechaza usuarios no admin. 9. PATCH /internal/flight-classes/:id/decrement-seats rechaza llamadas sin x-internal-api-key. 10. Gateway /health reporta servicios arriba/degradados.

10. Eventos y auditoria

Eventos de dominio actuales:

Evento	Productor	Disparador
ReservationCreated	booking-service	Reserva creada
ReservationCancelled	booking-service	Reserva cancelada
SeatAssigned	booking-service	Asiento asignado
CheckInCompleted	booking-service	Boarding pass creado
PaymentRegistered	payments-service	Pago registrado
InvoiceIssued	payments-service	Factura emitida
AncillaryAdded	payments-service	Servicio extra agregado

Envelope usado por el bus:

```
{
  "eventId": "uuid",
  "eventType": "ReservationCreated",
  "eventVersion": 1,
  "occurredAt": "2026-05-26T16:00:00.000Z",
  "producer": "booking-service",
  "correlationId": "cid",
  "payload": {}
}
```

Estado:

- Implementado como `EventEmitter` in-process.
- Util para evidencia academica de EDA y auditoria local.
- No reemplaza todavia un message broker real.

Recomendacion:

- Para una integracion real con Booking Prototipo, publicar estos eventos en un broker externo o exponerlos mediante webhook/event API versionada.

11. Despliegue

Comando esperado:

```
docker compose up --build
```

Servicios expuestos:

Servicio	URL local
API Gateway	http://localhost:3000
Auth Service	http://localhost:3001
Catalog Service	http://localhost:3002
Flights Service REST	http://localhost:3003
Flights Service gRPC	localhost:50051
Booking Service	http://localhost:3004
Payments Service	http://localhost:3005
Admin Service	http://localhost:3006
Jaeger UI	http://localhost:16686

Variables requeridas:

- JWT_SECRET
- JWT_ISSUER
- JWT_AUDIENCE
- JWT_EXPIRES_IN
- INTERNAL_API_KEY
- AUTH_DATABASE_URL
- CATALOG_DATABASE_URL
- FLIGHTS_DATABASE_URL
- BOOKING_DATABASE_URL
- PAYMENTS_DATABASE_URL
- ADMIN_DATABASE_URL
- FRONTEND_URL
- OTEL_EXPORTER_OTLP_ENDPOINT

12. Checklist de evidencia para entregar

Para sustentar el Reto 2 hasta semana 11, conviene anexar:

- Captura de `docker compose ps` con los servicios levantados.
- Captura de `/health` del API Gateway mostrando estados de microservicios.
- Captura de Jaeger con trazas entre Gateway y servicios.
- Captura de Swagger/OpenAPI en `/api/v1/docs`.
- Ejemplo de request con `X-Correlation-Id`.
- Ejemplo de request rechazado por JWT faltante.
- Ejemplo de request rechazado por `x-internal-api-key` faltante.
- Resultado de pruebas automatizadas.
- Evidencia de contract test o, si aun no existe, matriz de contratos por endpoint.

13. Brechas tecnicas antes de defensa

Prioridad alta:

- Corregir `vuelos-angular/src/app/app.spec.ts` o reemplazarlo por pruebas reales del layout/routing actual.
- Agregar suite de pruebas backend y gateway.

- Agregar contract tests para endpoints que consumira Booking Prototipo.
- Actualizar `.env.example` con variables de microservicios.

Prioridad media:

- Alinear `myReservations` de GraphQL con `/api/v1/reservations/my`.
- Agregar health checks de Docker por servicio.
- Crear pruebas gRPC para los contratos `.proto`.
- Separar completamente dependencias de dominio en el `admin-service` mediante clientes HTTP/gRPC en lugar de varios Prisma clients directos.

Prioridad futura:

- Reemplazar EventBus in-process por broker real.
- Agregar service mesh si se requiere mTLS, retries y politicas fuera del codigo.
- Publicar contratos en una carpeta o registry versionado, por ejemplo `contracts/rest`, `contracts/graphql`, `contracts/proto`, `contracts/events`.

14. Resumen por semana

Semana	Tema	Evidencia del proyecto
7	Esquemas, compatibilidad y contratos	<code>/api/v1</code> , OpenAPI, GraphQL schema, <code>.proto</code> , aliases backward compatible
8	API Gateway y Service Mesh	<code>api-gateway</code> , registry, proxy REST, GraphQL aggregator, circuit breaker
9	Pruebas de integracion y contract testing	Evidencia insuficiente; solo existe spec Angular y falla
10	Observabilidad y trazas distribuidas	OpenTelemetry, Jaeger, logs JSON, correlation ID
11	Seguridad en integraciones	JWT, roles, ownership, internal API key, Helmet, CORS, rate limiting

Conclusion: la arquitectura de Reto 2 esta mayormente implementada a nivel de microservicios, Gateway, contratos, observabilidad y seguridad. La brecha principal para semana 9 es la ausencia de una suite de pruebas de integracion y contract testing; ademas, el unico test existente en frontend esta desactualizado y no pasa actualmente.